UADC PoC Documentation

Generated from repository Markdown sources. Paths are repository-relative unless otherwise stated.

README

UADC PoC Collaboration Workspace

This workspace is for the xBRL-GL Next / UADC proof of concept.

The first PoC checkpoint is the EN 16931-1 invoice semantic model represented as an LHM/HMD-style CSV, plus binding-driven conversion from UBL Invoice XML to structured CSV, xBRL-CSV JSON metadata, and round-trip UBL Invoice XML.

OpenPeppol BIS Billing is handled as the next layer: a CIUS/profile overlay on top of EN 16931-1, with additional constraints, defaults, and syntax-specific rules.

Flow

```
UBL Invoice XML
  -> EN 16931 syntax binding
  -> EN 16931 LHM/HMD structured CSV
  -> xBRL-CSV JSON metadata
  -> Arelle xBRL-CSV validation
  -> round-trip UBL Invoice XML
  -> UBL 2.1 schema validation

OpenPeppol BIS CIUS
  -> profile constraints and extensions
  -> checks layered on the EN 16931 conversion baseline
```

Directory Layout

- `docs/` - collaboration notes and decisions.
- `references/` - external source notes and links.
- `specs/lhm/` - LHM/HMD semantic model definitions.
- `specs/bindings/` - syntax and semantic binding definitions. The active UBL Invoice syntax binding is `specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv`.
- `samples/input/` - sample input XML or CSV.
- `samples/expected/` - checked expected output.
- `src/` - PoC conversion and binding scripts.
- `tests/` - regression checks.
- `out/` - generated local output, ignored by Git.

The taxonomy generator is included at `tools/taxonomy/xBRLGL_TaxonomyGenerator.py`; no external `XBRL-GL-2026` checkout is required for normal tests. The generated xBRL-CSV taxonomy entry point is `out/taxonomy/plt/plt-oim-2026-07-05.xsd`. Tuple/content taxonomy schemas such as `plt-all-<version>.xsd` and `en16931-content-<version>.xsd` are not generated for this PoC.

Current Scope

1. Define and audit the Invoice LHM from EN 16931-1.
2. Confirm syntax-binding conversion from UBL Invoice XML to EN 16931

structured CSV.

3. Generate xBRL-CSV metadata that references the `plt-oim` taxonomy entry point.
4. Validate generated xBRL-CSV metadata with Arelle.

5. Reconstruct UBL Invoice XML from structured CSV and validate it with UBL 2.1 schemas.
6. Layer OpenPeppol BIS Billing as CIUS/profile constraints and extensions after the EN 16931 baseline remains stable.

Current Tests

```
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe' tests\test_xbrlgl_generator_uadc_lhm.py
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe' tests\validate_taxonomy.py
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe' tests\test_openpeppol_invoice_conversion.py
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe'
tests\test_bis_billing3_examples_conversion.py
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe' tests\test_roundtrip_artifacts.py
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe' tests\test_xbrl_csv_metadata_arelle.py
```

The sample UBL Invoice XML and BIS Billing examples are converted using `specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv`. OpenPeppol CIUS checks are added after the EN 16931 conversion baseline is stable.

Semantic path elements are generated from Business Terms using `lowerCamelCaseConcatenated`, for example:

```
Invoice issue date -> invoiceIssueDate
Seller postal address -> sellerPostalAddress
```

LHM Generation Program Specification

Program Specification: LHM Generation

1. Purpose

This document specifies the programs used to generate and validate the EN 16931 invoice Logical Hierarchical Model (LHM) CSV for the UADC Proof of Concept.

All paths in this document are relative to the `UADC_PoC` working directory after the repository is pushed or cloned.

2. Scope

The current baseline is the EN 16931-1 invoice semantic model. OpenPeppol BIS Billing is handled later as a CIUS/profile overlay.

Controlled source:

```
specs/lhm/source/EN16931_CIUS_Invoice_LHM_Source.csv
```

Generated LHM:

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

3. Components

3.1 Source-to-LHM builder

Program:

```
tools/build_lhm_from_source.py
```

Responsibilities:

- create an editable source CSV from an existing LHM CSV with `init-source`;
- generate the normalized LHM CSV from the editable source CSV with `build`;
- derive lowerCamelCaseConcatenated semantic path segments from Business Term values;
- derive `class_term` from the nearest parent BG Business Term, singularized;
- derive `lhm_level` for Structured CSV and xBRL-CSV taxonomy modeling;
- generate unique UpperCamelCase `element` values when no override is provided;
- preserve manual override columns.

3.2 Class and element normalizer

Program:

```
tools/normalize_lhm_class_element.py
```

Responsibilities:

- normalize `class_term`;
- normalize `element`;
- ensure generated element values are unique.

Rules:

- BG rows use their own Business Term as `class_term`, singularized.
- BT rows use the nearest parent BG Business Term as `class_term`, singularized.
- `element` is generated from `semantic_path`.
- `element` starts with an uppercase letter.
- If final semantic path segments duplicate, the shortest unique semantic path suffix is used.

3.3 Class and element checker

Program:

```
tools/check_lhm_class_element.py
```

Responsibilities:

- report `class_term` mismatches;
- report `element` mismatches;
- return a non-zero exit code when mismatches are found.

3.4 PDF definition updater

Program:

```
tools/update_lhm_definitions_from_pdf.py
```

Responsibilities:

- extract EN 16931-1 Table 2 Description cells with `pdfplumber`;
- update the LHM `definition` column for extracted BT/BG identifiers;
- apply built-in overrides for rows where PDF extraction is known to split cells;
- report unresolved empty definitions.

This utility updates the specified LHM CSV.

3.5 UBL syntax sequence updater

Program:

```
tools/update_lhm_syntax_sequence_from_ubl_xsd.py
```

Responsibilities:

- read extracted OASIS UBL 2.1 XSD files;
- resolve each LHM XPath against the UBL Invoice schema sequence;
- write `syntax_sequence` values that can be used for XML-order checks;
- keep the downloaded UBL schema package outside version control, normally under `out/cache`.

4. Editable Source CSV

Fields:

```
sequence
syntax_sequence
id
level
type
cardinality
business_term
description
usage_note
req_id
semantic_data_type
path
xpath
semantic_path_override
class_term_override
element_override
label_local
definition_local
source_ref
adjustment_note
```

Important fields:

- `sequence`: EN 16931-1 Table 2 order.
- `syntax_sequence`: UBL Invoice XML schema order, when populated from OASIS UBL 2.1 XSD.

- `id`: EN 16931 identifier such as `BG-4` or `BT-27`.
- `level`: hierarchy level. Invoice is level `0`; `BT-1` is level `1`.
- `cardinality`: source cardinality. Values ending in `..n` are normalized to `..*`.
- `business_term`: EN 16931 Business Term.
- `description`: EN 16931 Description. This becomes the LHM `definition`.
- `path`: slash-separated identifier path used to locate the parent BG.
- `xpath`: syntax binding reference path for UBL Invoice where available.
- `semantic_path_override`: optional full semantic path override.
- `class_term_override`: optional class term override.
- `element_override`: optional element name override.

5. Generated LHM CSV

Fields:

```
sequence
syntax_sequence
level
lhm_level
type
identifier
name
datatype
multiplicity
domain_name
definition
module
class_term
id
path
semantic_path
label_local
definition_local
element
xpath
```

Mapping rules:

- `level` preserves the EN 16931/LHM logical hierarchy.
- `lhm_level` is the effective hierarchy used by Structured CSV and xBRL-CSV taxonomy generation.
- `BG-ROOT` has `lhm_level=0`.
- A BG with multiplicity `0..*` or `1..*` has `lhm_level` equal to the nearest ancestor BG with an `lhm_level` plus `1`.
- A BG with multiplicity `0..1` or `1..1` has blank `lhm_level`, except `BG-ROOT`.
- A BT has `lhm_level` equal to the nearest ancestor BG with an `lhm_level` plus `1`.
- Blank `lhm_level` BG rows are retained in the semantic model but are not emitted as Structured CSV dimensions or xBRL-CSV dimension concepts.
- `name` is copied from `business_term`.
- `syntax_sequence` is copied from the source CSV or populated by the UBL syntax sequence updater.
- `datatype` is copied from `semantic_data_type`.
- `multiplicity` is copied from `cardinality`.
- `definition` is copied from `description`.
- `module` is currently `en16931`.
- `semantic_path` is either `semantic_path_override` or a generated path.
- `element` is either `element_override` or a generated unique UpperCamelCase name.

Semantic path rules:

- Business Terms are converted to lowerCamelCaseConcatenated path segments.
- The semantic path starts at `$.invoice`.
- `BG-0` is not generated because EN 16931-1 does not define it.

Multiplicity rules:

- LHM `multiplicity` must be one of `0..1`, `0..*`, `1..1`, or `1..*`.
- Source cardinalities `0..n` and `1..n` are normalized to `0..*` and `1..*`.
- Other multiplicity values are rejected during LHM generation.

Element name uniqueness rules:

7. Split `semantic_path` into path segments after removing the leading `$.`.
8. Use the final segment as the first candidate.
9. Convert the candidate segment or suffix to UpperCamelCase.
10. If that name is unique, use it as `element`.
11. If it duplicates another row, expand the candidate one segment to the left and try again.
12. Continue until the shortest unique semantic path suffix is found.
13. If no suffix is unique, append the row identifier without hyphens as a final fallback.

Example:

```
$.invoice.precedingInvoiceReference.precedingInvoiceReference
```

The final segment alone would produce `PrecedingInvoiceReference`, which also belongs to the BG row. The generator therefore expands the suffix and produces:

```
PrecedingInvoiceReferencePrecedingInvoiceReference
```

Example:

```
BT-1 Invoice number
semantic_path = $.invoice.invoiceNumber
class_term = Invoice
element = InvoiceNumber
level = 1
```

6. Validation Rules

The LHM checks verify that:

- semantic paths use lowerCamelCaseConcatenated path segments;
- `BG-0` is not present;
- `BG-ROOT` represents Invoice at level `0`;
- `BT-1` is `$.invoice.invoiceNumber` at level `1`;
- LHM element names are unique;
- multiplicity values are limited to `0..1`, `0..*`, `1..1`, and `1..*`;
- BG dimension columns are left aligned in hierarchical CSV output;
- non-repeating BGs such as Seller and Buyer are not emitted as dimension columns;
- repeating BGs such as Invoice Line are emitted as dimension columns.

7. Dependencies

Required:

- Python 3.10 or later.

Optional:

- `pdfplumber`, only for `tools/update_lhm_definitions_from_pdf.py`.

8. Non-Goals

The LHM generation programs do not:

- publish generated `out/` files to GitHub;
- fully model OpenPeppol BIS Billing profile constraints;

- validate XBRL taxonomy output.

LHM Generation User Guide

User Guide: LHM Generation

1. Working Directory

Run commands from the `UADC_PoC` directory:

```
cd UADC_PoC
```

All paths below are relative to this directory.

Set the Python command for the local Windows environment:

```
$python = 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe'
```

2. Edit the LHM Source

Edit:

```
specs/lhm/source/EN16931_CIUS_Invoice_LHM_Source.csv
```

Commonly edited columns:

- `description`
- `path`
- `xpath`
- `semantic_path_override`
- `class_term_override`
- `element_override`
- `label_local`
- `definition_local`
- `adjustment_note`

Leave override columns blank when generated values are acceptable.

Use only these multiplicity values in the generated LHM:

```
0..1
0..*
1..1
1..*
```

If source material uses `0..n` or `1..n`, the LHM generator normalizes those values to `0..*` or `1..*`.

The generated LHM has both `level` and `lhm_level`:

- `level` preserves the EN 16931/LHM logical hierarchy.
- `lhm_level` is the effective hierarchy for Structured CSV and xBRL-CSV taxonomy generation.
- `0..1` and `1..1` BG rows normally have blank `lhm_level` and are not emitted as dimensions.
- BT rows under those BGs use the nearest ancestor BG with an `lhm_level` plus `1`.
- Repeating BG rows, `0..*` and `1..*`, receive their own `lhm_level` and become dimensions.

3. Generate the LHM CSV

Run:

```
& $python .\tools\build_lhm_from_source.py build `
  .\specs\lhm\source\EN16931_CIUS_Invoice_LHM_Source.csv `
  .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv
```

Expected output:

```
Wrote generated LHM CSV: specs\lhm\EN16931_CIUS_Invoice_LHM.csv
```

4. Optional: Initialize the Source CSV

Use this only when a new editable source CSV must be created from an existing LHM CSV:

```
& $python .\tools\build_lhm_from_source.py init-source `
  .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv `
  .\specs\lhm\source\EN16931_CIUS_Invoice_LHM_Source.csv
```

This command rewrites the source CSV. Use it carefully.

5. Optional: Normalize Class and Element Values

Prefer rebuilding from the source CSV. If the generated LHM CSV has been edited directly, normalize it with:

```
& $python .\tools\normalize_lhm_class_element.py `
  .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv
```

6. Optional: Populate UBL Syntax Sequence

Download and extract the official OASIS UBL 2.1 package into a local, non-versioned directory such as:

```
out/cache/UBL-2.1
```

Then populate `syntax_sequence` values from the UBL Invoice schema:

```
& $python .\tools\update_lhm_syntax_sequence_from_ubl_xsd.py `
  .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv `
  --schema-root .\out\cache\UBL-2.1\xsd `
  -o .\out\cache\EN16931_CIUS_Invoice_LHM.syntax_sequence_check.csv
```

Use the generated file to review XML schema order. The EN 16931 `sequence` column remains the semantic Table 2 order; `syntax_sequence` is the UBL XML order used for XML-oriented checks and reverse output ordering.

7. Optional: Update Definitions from PDF

If the EN 16931-1 PDF is available and `pdfplumber` is installed:

```
& $python .\tools\update_lhm_definitions_from_pdf.py `
  "<path-to-EN16931-1-pdf>" `
  .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv `
  --first-page 43 `
  --last-page 75
```

This updates only the CSV file passed on the command line.

8. Check the LHM

Run semantic path checks:

```
& $python .\tests\test_lhm_semantic_paths.py
```

Run class and element checks:

```
& $python .\tools\check_lhm_class_element.py `
  .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv
```

Run hierarchical CSV layout checks:

```
& $python .\tests\test_lhm_hierarchical_csv_layout.py
```

9. Troubleshooting

Duplicate element names

Regenerate the LHM from the source CSV. Element names are generated from `semantic_path` by using the shortest unique suffix. If a duplicate remains intentional, set `element_override` in the source CSV.

Wrong class term

Check the source CSV `path` column. BT rows use the nearest parent BG found in the path.

Wrong semantic path

Set `semantic_path_override` in the source CSV and rebuild the LHM.

Missing PDF descriptions

Check the PDF page range. Some rows may need manual `description` values in the source CSV because PDF table extraction can split cells.

Syntax Binding Conversion Program Specification

Program Specification: Syntax Binding XML-to-Hierarchical-CSV Conversion

1. Purpose

This document specifies the syntax binding conversion program used by the UADC Proof of Concept.

The program converts an XML invoice document into a dimension-based hierarchical CSV by applying a syntax binding CSV. During forward conversion it also writes JSON metadata that relates the CSV columns to the generated taxonomy. It can also reverse a hierarchical CSV back to XML with the same binding information.

All paths in this document are relative to the `UADC_PoC` working directory after the repository is pushed or cloned, except paths beginning with `../`, which refer to sibling directories of `UADC_PoC`.

2. Main Program

Program:

```
src/syntax_binding_hierarchical.py
```

Supporting XML extraction module:

```
src/syntax_binding.py
```

The hierarchical converter imports namespace and XPath helper functions from `syntax_binding.py`.

3. Input Files

3.1 XML input

The XML input is a UBL Invoice XML document.

PoC sample:

```
samples/input/openpeppol_ubl_invoice_minimal.xml
```

Package sample:

```
../syntax_binding_revised_package/invoice.xml
```

3.2 Syntax binding CSV

The binding CSV maps semantic model paths to XML XPath expressions.

PoC binding:

```
specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv
```

Package binding:

```
../syntax_binding_revised_package/bindings.csv
```

Required binding columns:

```
semantic_path
xpath
```

The parser also accepts compatible column names:

```
path
source_xpath
xml_path
```

Rows without both semantic path and XPath are ignored.

3.3 LHM CSV

When `--lhm-csv` is supplied, the converter uses the LHM to define:

- output column order;
- BG dimension columns;
- BT value columns;
- mapping from semantic paths to final LHM `element` names;
- repeating BGs based on `multiplicity`;
- repeat context XPath for BG rows where available.

PoC LHM:

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

3.4 Template CSV

When `--template-csv` is supplied, the converter uses its header as the output column order and infers some field-to-dimension placement from non-empty template rows.

Package template:

```
../syntax_binding_revised_package/invoice.csv
```

If both `--lhm-csv` and `--template-csv` are supplied, the LHM layout controls the main field order and the template may still provide field placement hints.

4. Forward Output

The output is a UTF-8-SIG CSV file.

PoC output:

```
out/hierarchical/en16931_lhm_hierarchical.csv
```

Package output:

```
out/hierarchical/package_invoice_hierarchical.csv
```

The output contains:

- `dInvoice` as the root dimension column;
- additional `d*` columns for repeating BGs;
- BT value columns after dimension columns;
- one root row for invoice-level facts;
- one row per repeated BG context where bindings produce values.

Forward conversion also writes JSON metadata. By default the metadata file name is:

```
<output-csv>.metadata.json
```

For example:

```
out/hierarchical/en16931_lhm_hierarchical.csv.metadata.json
```

The metadata is an xBRL-CSV metadata document that Arelle can load with the `loadFromOIM` plugin. It links the structured CSV table to the generated xBRL-CSV taxonomy entry points and maps CSV columns to taxonomy concepts. The report `documentInfo.taxonomy` property must not be empty; missing `plt-oim` is treated as a conversion error. The JSON metadata names the xBRL-CSV OIM taxonomy entry point `out/taxonomy/plt/plt-oim-<version>.xsd`.

5. Reverse Output

When `--reverse` is used, the input is a hierarchical CSV and the output is an XML file.

Reverse output example:

```
out/reverse/en16931_reverse_invoice.xml
```

The reverse converter:

- reads the hierarchical CSV rows;

- resolves fields through the syntax binding CSV;
- uses the LHM layout to associate fields with dimension rows;
- creates UBL Invoice XML elements from bound XPath expressions;
- creates one XML context for each repeated dimension row that has bound values.
- derives required `currencyID` attributes for amount elements from invoice currency fields.

6. Command-Line Interface

```
syntax_binding_hierarchical.py XML_FILE -b BINDING_CSV -o OUTPUT_CSV [options]
```

Reverse form:

```
syntax_binding_hierarchical.py INPUT_CSV --reverse -b BINDING_CSV -o OUTPUT_XML [options]
```

Arguments:

- `XML_FILE`: input XML file in forward mode.
- `INPUT_CSV`: input hierarchical CSV file in reverse mode.
- `-b`, `--binding`: syntax binding CSV.
- `-o`, `--output`: output hierarchical CSV in forward mode, or output XML in reverse mode.
- `--template-csv`: optional CSV template defining column order and dimension placement.
- `--lhm-csv`: optional LHM CSV defining BG dimension columns and BT value columns.
- `--metadata-output`: optional JSON metadata output path. Default is output CSV path plus `.metadata.json`.
- `--taxonomy-base`: generated taxonomy base directory referenced by JSON metadata. Default is `out/taxonomy`.
- `--reverse`: convert hierarchical CSV back to XML.
- `--drop-empty-columns`: remove columns that have no values in the generated rows.
- `--d-invoice`: value written into the `dInvoice` column. Default is `1`.
- `-e`, `--encoding`: CSV encoding. Default is `utf-8-sig`.

7. Forward Processing Model

7.1 Namespace handling

The converter collects XML namespace declarations from the input XML using `xml.etree.ElementTree.iterparse`.

XPath helper functions support namespace-prefixed elements such as:

```
/Invoice/cbc:ID
/Invoice/cac:AccountingSupplierParty/cac:Party/cac:PartyName/cbc:Name
```

7.2 LHM layout loading

When an LHM CSV is provided:

14. C rows are treated as BG/class rows.
15. A C row becomes a dimension only if it is the invoice root or if `lhm_level` is populated.
16. Non-repeating BG rows normally have blank `lhm_level`; they remain semantic grouping nodes and are not emitted as dimensions.
17. Dimension names are generated as `d` + UpperCamelCase(`element`).
18. A rows are treated as BT/fact rows.
19. Fact column names are taken from the LHM `element` column.
20. Each fact is assigned to the nearest ancestor dimension, equivalent to the nearest ancestor BG with an `lhm_level`.

Repeating multiplicity is detected when the upper bound is:

```
*
n
unbounded
```

or a numeric value greater than `1`.

7.3 Binding parsing

Each binding row is converted into an internal binding record:

```
semantic_path
xpath
root
dimension
filter_field
filter_value
field
```
The converter supports:

- root-level paths such as `$.invoice.invoiceNumber`;
- dimension paths with optional filter predicates;
- generic nested semantic paths resolved through the nearest LHM dimension.

7.4 Row generation

The converter creates:

- a root invoice row for invoice-level facts;
- dimension rows for filtered dimension facts;
- repeated BG rows for repeated contexts such as VAT breakdown or invoice line.

For repeated BG rows:

21. Bindings are grouped by dimension.
22. The repeat XPath is taken from the LHM BG row XPath when available.
23. Otherwise the repeat XPath is inferred from common binding XPath prefixes.
24. One output row is written for each matched XML context.
25. Ancestor repeating dimension values are copied into nested repeated rows.

7.5 Column handling

If the LHM is used, field order is:

```
dimension columns first, then fact columns
```
If a template is used without LHM, the template header controls the output order.

If neither LHM nor template provides a header, field names are derived from bindings.

8. Metadata JSON Processing Model

In forward mode the converter writes JSON metadata after the structured CSV is generated.

The metadata structure follows the xBRL-CSV report package shape:

- `documentInfo`: xBRL-CSV document type, namespaces, and taxonomy entry point references.
- `tables`: the generated structured CSV table and its relative URL.
- `tableTemplates`: table-level dimensions and per-column concept mappings.

Dimension columns are declared as table dimensions:

```
"plt:d_en16931_Invoice": "$dInvoice"
```
Fact columns are declared with xBRL concepts:

Example:

```
{
  "InvoiceNumber": {
    "dimensions": {
      "concept": "en16931:InvoiceNumber"
    }
  }
}
```
Amount facts receive a default test unit of `iso4217:JPY` in metadata when the LHM datatype indicates an amount. Production use should derive the unit from the applicable currency term.

9. Reverse Processing Model

In reverse mode:

26. The converter reads the input hierarchical CSV with `csv.DictReader`.
27. The converter reads the LHM layout and binding CSV in the same way as forward mode.
28. Root-level fields are written to XML paths directly below the invoice root.
29. Dimension fields are grouped by their `d*` dimension column.
30. For each dimension row with bound values, the converter creates a repeated XML context from the BG XPath.
31. Bound field values are written to relative XPaths under that context.
32. XPath predicates such as `cbc:ChargeIndicator=false()` and `cbc:DocumentTypeCode='130'` are materialized as child values when a matching XML context is created.
33. Amount elements receive `currencyID` from the invoice document currency field. Tax accounting currency TaxTotal paths receive `currencyID` from the tax accounting currency field.
34. UBL-required supporting elements that are not EN 16931 BT values are added where needed for schema validation, including `cac:TaxScheme/cbc:ID` and missing `cbc:ChargeIndicator=false`.
35. Generated UBL child elements are normalized to the UBL 2.1 sequence for the supported Invoice structures.

The reverse converter currently targets the PoC UBL Invoice binding pattern. It is intended for round-trip verification of the structured CSV representation rather than canonical XML reproduction.

10. Currency Attribute Rules

The syntax binding treats `currencyID` as syntax metadata derived from semantic currency terms:

- BT-5 `DocumentCurrencyCode` is the default `currencyID` for invoice amount elements.
- BT-6 `TaxAccountingCurrencyCode` is the `currencyID` for the tax accounting currency TaxTotal branch.
- BT-110 and BG-23 use the UBL path predicate `cbc:TaxAmount/@currencyID=/Invoice/cbc:DocumentCurrencyCode`.
- BT-111 uses the UBL path predicate `cbc:TaxAmount/@currencyID=/Invoice/cbc:TaxCurrencyCode`.

The CSV therefore stores semantic amount values and currency code values separately. Reverse conversion writes the required UBL `currencyID` attributes.

11. Constraints

- The converter does not validate the XML against UBL schemas internally; the regression test `tests/test_roundtrip_xml_ubl_schema.py` performs UBL 2.1 schema validation.
- The converter does not validate EN 16931 business rules.
- The converter does not run Arelle internally; the regression test `tests/test_xbrl_csv_metadata_arelle.py` validates generated metadata with Arelle.
- The converter does not evaluate full XPath 2.0.
- Supported predicates are intentionally limited to the cases used by the PoC bindings.
- Template columns alone do not create rows; rows require matching bindings.
- Reverse XML element order follows LHM `syntax_sequence` and the supported UBL 2.1 child-order normalization table.
- Reverse XML may omit unbound XML content.
- Reverse XML is generated from bound CSV values and is not intended to be byte-for-byte identical to the source XML.

12. Regression Tests

PoC LHM-driven conversion test:

```
tests/test_lhm_hierarchical_csv_layout.py
```

Package template/binding conversion test:

```
tests/test_syntax_binding_hierarchical.py
```

OpenPeppol structured conversion test:

`tests/test_openpeppol_invoice_conversion.py`
Reverse conversion round-trip test:

`tests/test_syntax_binding_reverse.py`
Round-trip artifact and metadata test:

`tests/test_roundtrip_artifacts.py`

13. Non-Goals

The syntax binding converter does not:

- generate LHM definitions;
- generate XBRL taxonomy files;
- run semantic binding to flat CSV;
- perform full EN 16931 or OpenPeppol validation;
- publish generated `out/` files to GitHub.

Syntax Binding Conversion User Guide

User Guide: Syntax Binding XML-to-Hierarchical-CSV Conversion

1. Working Directory

Run commands from the `UADC_PoC` directory:

```
cd UADC_PoC
```

All paths below are relative to this directory unless they begin with `../`.

Set the Python command for the local Windows environment:

```
$python = 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe'
```

2. Main Script

Use:

```
src/syntax_binding_hierarchical.py
```

This script converts XML to dimension-based hierarchical CSV using a syntax binding CSV. It also supports reverse conversion from hierarchical CSV back to XML with `--reverse`.

3. Convert the PoC OpenPeppol Sample

Run:

```
& $python .\src\syntax_binding_hierarchical.py `
  .\samples\input\openpeppol_ubl_invoice_minimal.xml `
  -b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
  --lhm-csv .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv `
  -o .\out\hierarchical\en16931_lhm_hierarchical.csv
```

Output:

```
out/hierarchical/en16931_lhm_hierarchical.csv
out/hierarchical/en16931_lhm_hierarchical.csv.metadata.json
```

This command uses the LHM CSV to determine the output dimensions and fact columns. It also creates JSON metadata that relates the CSV columns to the generated taxonomy.

The converter treats `lhm_level` as the effective hierarchy for Structured CSV. BG rows with blank `lhm_level`, such as non-repeating Seller or Buyer groups, are semantic grouping nodes only; their BT values are written in the nearest ancestor dimension row.

4. Convert the Revised Package Sample

Run:

```
& $python .\src\syntax_binding_hierarchical.py `
  ..\syntax_binding_revised_package\invoice.xml `
  -b ..\syntax_binding_revised_package\bindings.csv `
  --template-csv ..\syntax_binding_revised_package\invoice.csv `
  -o .\out\hierarchical\package_invoice_hierarchical.csv
```

Output:

```
out/hierarchical/package_invoice_hierarchical.csv
out/hierarchical/package_invoice_hierarchical.csv.metadata.json
```

This command uses the package template CSV to preserve the expected output column order.

5. Command Options

Basic form:

```
& $python .\src\syntax_binding_hierarchical.py XML_FILE `
  -b BINDING_CSV `
  -o OUTPUT_CSV
```
Options:

- `--lhm-csv`: use an LHM CSV to define dimension columns and BT value columns.
- `--template-csv`: use a template CSV header to define output column order.
- `--metadata-output`: set the JSON metadata output path. Default is `OUTPUT_CSV.metadata.json`.
- `--taxonomy-base`: set the generated taxonomy directory referenced by JSON metadata. Default is `out/taxonomy`.
- `--reverse`: convert hierarchical CSV back to XML.
- `--drop-empty-columns`: remove output columns that have no values.
- `--d-invoice`: set the `dInvoice` value. Default is `1`.
- `-e`, `--encoding`: CSV encoding. Default is `utf-8-sig`.

6. Reverse Hierarchical CSV to XML

First create or confirm the hierarchical CSV:

```
& $python .\src\syntax_binding_hierarchical.py `
  .\samples\input\openpeppol_ubl_invoice_minimal.xml `
  -b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
  --lhm-csv .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv `
  -o .\out\hierarchical\en16931_lhm_hierarchical.csv
```
Then reverse it to XML:

```
& $python .\src\syntax_binding_hierarchical.py `
  .\out\hierarchical\en16931_lhm_hierarchical.csv `
  --reverse `
  -b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
  --lhm-csv .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv `
  -o .\out\reverse\en16931_reverse_invoice.xml
```
Output:

```
out/reverse/en16931_reverse_invoice.xml
```
The reverse XML is generated from bound CSV values. It is intended for round-trip verification and may not reproduce unbound XML content. When the LHM has `syntax_sequence` values, reverse output uses them to follow UBL schema order.

Amount `currencyID` attributes are derived during reverse conversion:

- `DocumentCurrencyCode` is used for ordinary invoice amount elements.
- `TaxAccountingCurrencyCode` is used for the tax accounting currency TaxTotal branch.

7. Generate Test Artifacts with Metadata

Round-trip test artifacts are built under:

```
tests/roundtrip/
```
Run:

```
& $python .\tests\build_roundtrip_test_artifacts.py
```
The generated layout is:

```
tests/roundtrip/<dataset>/
  source_xml/
  structured_csv/
  metadata_json/
  roundtrip_xml/
```
Meaning:

- `source_xml`: source UBL Invoice XML.
- `structured_csv`: hierarchical structured CSV.
- `metadata_json`: JSON metadata relating the CSV columns to the taxonomy.

- `roundtrip_xml`: XML regenerated from the structured CSV.

The script uses `--metadata-output` to place metadata JSON in `metadata_json/`.

8. Input Binding CSV

The binding CSV should contain:

```
semantic_path,xpath
```

Example:

```
$.invoice.invoiceNumber,/Invoice/cbc:ID
$.invoice.invoiceIssueDate,/Invoice/cbc:IssueDate
```

Compatible column aliases are also accepted:

```
path
source_xpath
xml_path
```

9. Output CSV Layout

The hierarchical CSV uses:

- `dInvoice` for the invoice root;
- `d*` columns for repeated BG dimensions;
- fact columns for BT values.

When the LHM is used, dimension columns are placed first and BT columns follow.

Example output pattern:

```
dInvoice,dVatBreakdown,dInvoiceLine,InvoiceNumber,InvoiceIssueDate,...
1,,,INV-2026-0001,2026-07-06,...
1,1,,,,...
1,,1,,,...
```

10. JSON Metadata Layout

The metadata file is an xBRL-CSV metadata document. It has these main sections:

- `documentInfo`: xBRL-CSV document type and taxonomy references.
- `tables`: the generated structured CSV table URL.
- `tableTemplates`: table-level dimensions and fact concept mappings.

Check that important columns are mapped as expected:

```
tableTemplates.structured.dimensions["plt:d_en16931_Invoice"] -> "$dInvoice"
tableTemplates.structured.columns.InvoiceNumber.dimensions.concept -> "en16931:InvoiceNumber"
tableTemplates.structured.columns.DocumentCurrencyCode.dimensions.concept -> "en16931:DocumentCurrencyCode"
```

11. Run Regression Checks

Run the LHM-driven OpenPeppol sample conversion check:

```
& $python .\tests\test_lhm_hierarchical_csv_layout.py
```

Run the revised package conversion check:

```
& $python .\tests\test_syntax_binding_hierarchical.py
```

Run the OpenPeppol structured conversion check:

```
& $python .\tests\test_openpeppol_invoice_conversion.py
```

Run the reverse conversion round-trip check:

```
& $python .\tests\test_syntax_binding_reverse.py
```

Run the round-trip artifact and metadata check:

```
& $python .\tests\test_roundtrip_artifacts.py
```
Validate the generated xBRL-CSV metadata with Arelle:

```
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```
Validate the regenerated round-trip XML against the UBL 2.1 Invoice schema:

```
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

12. Troubleshooting

No usable bindings found

Check that the binding CSV contains `semantic_path` and `xpath` values.

Missing output columns

If `--drop-empty-columns` is used, columns with no generated values are removed. Run without this option when a stable full header is required.

Metadata JSON cannot locate the taxonomy

The converter must write a non-empty `documentInfo.taxonomy` array. Generate the taxonomy first:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```
Or pass `--taxonomy-base` with the directory containing `plt/plt-oim-*.xsd`. If this schema is missing, metadata generation fails instead of writing an empty taxonomy list.

Repeating rows are missing

Check that:

- the BG row in the LHM has repeating multiplicity such as `0..*` or `1..*`;
- the BG row has an XPath that points to the repeated XML context;
- the syntax binding CSV contains bindings under that BG semantic path.

Values are written to the invoice root row instead of a dimension row

Use `--lhm-csv` so the converter can resolve each BT semantic path to its nearest LHM dimension.

Package sample line rows are empty

Template columns alone do not create rows. Invoice line rows require invoice line bindings.

Reverse XML does not match the original file order

Reverse XML is reconstructed from binding rows and hierarchical CSV values. Populate LHM `syntax_sequence` from the UBL schema when XML element order must be checked. Unbound XML content still cannot be reproduced until it is represented by a binding or fixed-value rule.

Taxonomy Generation Program Specification

Program Specification: Taxonomy Generation

1. Purpose

This document specifies the taxonomy generation program used by the UADC Proof of Concept.

All paths in this document are relative to the repository root after the repository is pushed or cloned. The taxonomy generator is included in this repository; it no longer depends on an external `XBRL-GL-2026` checkout.

2. Scope

Input LHM:

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

Taxonomy generator:

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

Output directory:

```
out/taxonomy/
```

The output directory is generated local output and is not intended to be pushed to GitHub.

3. Component

Program:

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

Responsibilities:

- read the UADC LHM CSV layout through `csv.DictReader`;
- normalize UADC LHM records into the internal XBRL-GL generator layout;
- generate module taxonomy schema files;
- generate an xBRL-CSV dimensional taxonomy schema;
- generate label, presentation, definition linkbase, JSON metadata, and CSV skeleton files.

The generator accepts UADC LHM CSV headers directly. It also supports older XBRL-GL LHM layouts where compatible.

4. Input Contract

The generator consumes the generated LHM CSV:

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

Required fields include:

```
sequence
level
lhm_level
type
name
datatype
multiplicity
definition
module
class_term
id
path
semantic_path
element
xpath
```

Important interpretation rules:

- `module` becomes the module namespace prefix, currently `en16931`.
- `element` becomes the concept name.
- `type` controls whether a row is treated as a field or group for dimensional modeling.
- `multiplicity` is used for dimensional modeling.
- `level`, `path`, and `semantic_path` preserve the logical LHM hierarchy.
- `lhm_level` defines the effective hierarchy for xBRL-CSV taxonomy generation.
- BG rows with blank `lhm_level` are retained as semantic groupings but are not emitted as hypercube, dimension, or primary item concepts.
- BT rows are attached to the cube of the nearest ancestor BG with an `lhm_level`.

Allowed `multiplicity` values are `0..1`, `0..*`, `1..1`, and `1..*`. The LHM generation step normalizes source values such as `0..n` and `1..n` before taxonomy generation.

5. Taxonomy Output

The EN 16931 PoC output includes:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label-ja.xml
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/plt-def-2026-07-05.xml
out/taxonomy/plt/plt-oim-2026-07-05.xsd
```

5.1 Module schema

File pattern:

```
out/taxonomy/<module>/<module>-<version>.xsd
```

For this PoC:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
```

Responsibilities:

- declare item concepts for LHM Field rows;
- assign `xbrli:periodType="instant"` to `substitutionGroup="xbrli:item"` elements.
- import the GL generic type schema from `../gen/gl-gen-<version>.xsd` when a GL generic type is used.

5.2 Omitted tuple/content schemas

No XBRL 2.1 tuple taxonomy entry point or tuple/content schema is generated for this Structured CSV PoC. In particular, these files are not defined:

- `out/taxonomy/plt/plt-all-<version>.xsd`
- `out/taxonomy/plt/<module>-content-<version>.xsd`

5.3 GL generic type schema

File pattern:

```
out/taxonomy/gen/gl-gen-<version>.xsd
```

Responsibilities:

- provide reusable GL item types such as `gen:amountItemType` and `gen:emailAddressItemType`;
- use the same version namespace as the generated taxonomy;
- be referenced from module schemas as `../gen/gl-gen-<version>.xsd`.

5.4 xBRL-CSV taxonomy schema

File pattern:

```
out/taxonomy/plt/plt-oim-<version>.xsd
```

Responsibilities:

- define `h_*` hypercube concepts using `xbrldt:hypercubeItem`;
- define `d_*` dimension concepts using `xbrldt:dimensionItem`;
- define `p_*` primary item concepts using `xbrli:item`;
- define the typed dimension domain element `_v`;
- define role types used by the xBRL-CSV definition linkbase.

Restrictions:

- `plt-oim` must not define tuple-supporting `complexType`.
- `plt-oim` must not define `xbrli:tuple` concepts.
- `plt-oim` must not import the module content schema.

5.5 Definition linkbase

File pattern:

```
out/taxonomy/plt/plt-def-<version>.xml
```

Responsibilities:

- reference `plt-oim` role types and dimensional concepts;
- connect primary items, hypercubes, and dimensions;
- represent BG-to-BG hierarchy using dimensional relationships for xBRL-CSV.

The definition linkbase locators for dimensional relationships point to `plt-oim`.

6. Validation Rules

The taxonomy regression checks verify that:

- `plt-oim` contains hypercube, dimension, and primary item definitions;
- `plt-oim` contains no `xbrli:tuple`;
- `plt-oim` contains no `complexType`;
- `plt-oim` does not import `en16931-content`;
- `plt-all` is not generated;
- `en16931-content` is not generated;
- module schemas import `../gen/gl-gen-<version>.xsd`;
- `xbrli:item` elements generated in `plt-oim` have `xbrli:periodType="instant"`.

7. Dependencies

Required:

- Python 3.10 or later.

8. Non-Goals

The taxonomy generation program does not:

- update LHM source CSV files;
- publish generated `out/` files to GitHub;
- validate the generated DTS with an external XBRL processor;
- validate xBRL-CSV instances beyond the regression checks.

Taxonomy Generation User Guide

User Guide: Taxonomy Generation

1. Working Directory

Run commands from the `UADC_PoC` directory:

```
cd UADC_PoC
```

All paths below are relative to this directory. The taxonomy generator is included in `tools/taxonomy/`, so no external `XBRL-GL-2026` checkout is required.

Set the Python command for the local Windows environment:

```
$python = 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe'
```

2. Input

The taxonomy generator uses:

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

Regenerate and check the LHM before generating taxonomy when the source model has changed.

3. Generate the Taxonomy

Run the taxonomy regression script:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

This regenerates files under:

```
out/taxonomy/
```

4. Direct Generator Command

To run the taxonomy generator directly:

```
& $python .\tools\taxonomy\xBRLGL_TaxonomyGenerator.py `
  .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv `
  -b .\out\taxonomy `
  -p en16931 `
  -n https://example.com/uada/en16931/invoice/2026-07-05 `
  -l ja `
  -c JPY
```

Useful options:

- `-b`, `--base_dir`: output directory.
- `-p`, `--palette`: palette/module prefix used for generated files.
- `-r`, `--root`: root element name for JSON metadata and CSV skeleton generation.
- `-l`, `--lang`: local label language.
- `-c`, `--currency`: ISO currency for amount facts.
- `-n`, `--namespace`: namespace. The final 10 characters are used as the version date.
- `-e`, `--encoding`: input/output encoding. Default is `utf-8-sig`.
- `-t`, `--trace`: print trace messages.
- `-d`, `--debug`: print debug messages.

5. Output Files

Key files:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label.xml
```

```
out/taxonomy/en16931/lang/en16931-2026-07-05-label-ja.xml
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/plt-def-2026-07-05.xml
out/taxonomy/plt/plt-oim-2026-07-05.xsd
```

Meaning:

- `en16931/en16931-2026-07-05.xsd`: module item concepts.
- `gen/gl-gen-2026-07-05.xsd`: GL generic item type schema referenced as `../gen/gl-gen-<version>.xsd`.
- `plt/plt-oim-2026-07-05.xsd`: xBRL-CSV taxonomy schema with hypercubes, dimensions, and primary items.
- `plt/plt-def-2026-07-05.xml`: xBRL-CSV dimensional definition linkbase.

6. Verify the Taxonomy Separation

Check that the local taxonomy structure is consistent:

```
& $python .\tests\validate_taxonomy.py
```

Expected output:

```
ok: local taxonomy checks passed
```

Check that `plt-oim` does not contain tuple or content schema definitions:

```
Select-String -Path .\out\taxonomy\plt\plt-oim-2026-07-05.xsd `
  -Pattern 'en16931-content','complexType','xbrli:tuple'
```

Expected result: no matches.

Check that `plt-oim` contains hypercube, dimension, and primary item concepts:

```
Select-String -Path .\out\taxonomy\plt\plt-oim-2026-07-05.xsd `
  -Pattern 'xbrldt:hypercubeItem','xbrldt:dimensionItem','substitutionGroup="xbrli:item"'
```

Check that the module schema imports the GL generic schema:

```
Select-String -Path .\out\taxonomy\en16931\en16931-2026-07-05.xsd `
  -Pattern '../gen/gl-gen-2026-07-05.xsd'
```

Check that `plt-all` and `en16931-content` are not generated:

```
Test-Path .\out\taxonomy\plt\plt-all-2026-07-05.xsd
Test-Path .\out\taxonomy\plt\en16931-content-2026-07-05.xsd
```

Expected result: `False`.

7. Run Arelle Taxonomy Validation

If Arelle is installed, run:

```
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\Scripts\arelleCmdLine.exe' `
  --file .\out\taxonomy\plt\plt-oim-2026-07-05.xsd `
  --validate
```

Expected output includes:

```
[info] validated
```

8. Run the Regression Check

Run:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

Expected output includes:

```
ok: XBRL-GL generator accepted UADC LHM CSV
```

9. Troubleshooting

Python is not found

Use the full Python path:

```
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe' --version
```

`plt-oim` contains tuple or content definitions

Regenerate with the current generator:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

The current design is xBRL-CSV only. It keeps hypercube, dimension, and primary item definitions in `plt-oim`; it does not generate XBRL 2.1 tuple concepts.

The taxonomy uses an unexpected date

Check the `-n` namespace option. The generator uses the final 10 characters of the namespace as the version date.

Decision 0001 Workspace Scope

Decision 0001: Workspace Scope

The PoC workspace uses OpenPeppol BIS Billing 3.0 upcoming UBL Invoice as the input syntax. The first semantic definition is an EN 16931 CIUS Invoice LHM/HMD CSV.

`../XBRL-GL-2026` remains a historical reference workspace for UN/CEFACT CCL to FSM/BSM/LHM/taxonomy experiments. The PoC taxonomy generator required for tests is now copied into `tools/taxonomy/`, so this repository has no runtime dependency on the external XBRL-GL-2026 workspace.

Decision 0002 EN16931 First OpenPeppol Overlay

Decision: EN 16931 First, OpenPeppol Overlay Second

Context

The PoC needs to confirm that an LHM/HMD-style semantic model and binding-driven conversion can represent the EN 16931-1 invoice model before adding profile specific behavior.

OpenPeppol BIS Billing is not just a syntax sample. It is a CIUS/profile layer over EN 16931 and can define additional constraints, default values, and syntax-specific requirements.

Decision

36. Keep the first checkpoint focused on EN 16931-1 LHM coverage and conversion.
37. Use UBL Invoice XML as a test input syntax for that checkpoint.
38. Treat OpenPeppol BIS Billing as a later overlay that adds CIUS/profile

constraints and OpenPeppol-specific binding checks.

Consequences

- `specs/lhm/EN16931_CIUS_Invoice_LHM.csv` is audited against EN 16931-1 BG/BT

identifiers.

- `specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv` is the current

conversion baseline.

- OpenPeppol-specific constraints should be documented and tested separately

after the EN 16931 baseline is stable.

Decision 0003 LHM Level Effective Hierarchy

Decision: Use `lhm_level` as the Effective Hierarchy

Context

The EN 16931 table level and the LHM logical level are useful for semantic review, but they are not always the best hierarchy for structured CSV rows or xBRL-CSV dimensions. In particular, BGs with `0..1` or `1..1` multiplicity such as Seller, Buyer, and their postal addresses add semantic grouping but do not need separate dimension columns.

Decision

39. Keep `level` as the EN 16931/LHM logical hierarchy.
40. Add and use `lhm_level` as the effective hierarchy for structured CSV and

taxonomy generation.

41. Leave `lhm_level` blank for non-repeating BGs that should not become

dimensions.

42. Assign BT rows to the nearest ancestor BG with an effective `lhm_level`.
43. Emit dimensions only for the invoice root and BG rows with populated

`lhm_level`.

Consequences

- Non-repeating BGs remain in the LHM for semantic review and XPath ownership.
- Structured CSV avoids unnecessary `dSeller`, `dBuyer`, and postal-address

dimension columns.

- Repeating BGs such as VAT breakdown and invoice line define stable dimension

columns and taxonomy dimensions.

- Taxonomy generation and XML conversion use the same effective hierarchy.

Decision 0004 xBRL-CSV Only Taxonomy

Decision: Generate xBRL-CSV Taxonomy Only

Context

Earlier experiments distinguished XBRL 2.1 tuple-oriented taxonomy output from xBRL-CSV taxonomy output. The UADC PoC structured CSV is intended to be validated as xBRL-CSV, so tuple concepts and tuple entry-point schemas add complexity without supporting the current checkpoint.

Decision

44. Generate the `plt-oim-<version>.xsd` xBRL-CSV taxonomy schema.
45. Generate hypercube, dimension, and primary item concepts in `plt-oim`.
46. Do not generate `en16931-content-<version>.xsd`; item declarations remain in the module schema and xBRL-CSV primary items remain in `plt-oim`.
47. Do not generate `plt-all-<version>.xsd`.
48. Do not define XBRL 2.1 tuple `complexType` structures for this PoC.
49. Reference `gl-gen` from generated module schemas as `../gen/gl-gen-<version>.xsd`.

Consequences

- The taxonomy output matches the structured CSV validation target.
- Arelle validation focuses on the xBRL-CSV DTS entry point.
- Tuple-specific implementation and tests are removed from the current scope.
- Future tuple taxonomy work, if needed, should be handled as a separate design

track.

Decision 0005 xBRL-CSV Metadata and Round Trip Validation

Decision: Use xBRL-CSV Metadata and Schema-Validated Round Trip XML

Context

The converter originally produced a UADC-specific JSON mapping file. That was useful during early development, but it could not be loaded directly as an xBRL-CSV report by Arelle. The PoC also needs proof that structured CSV can be reconstructed into UBL Invoice XML with schema-valid syntax.

Decision

50. Forward syntax binding conversion writes xBRL-CSV metadata JSON, not a

UADC-specific metadata format.

51. The metadata uses `documentInfo`, `tables`, and `tableTemplates`.
52. Metadata references the generated `plt-oim` taxonomy schema as the xBRL-CSV

OIM entry point.

53. Arelle `loadFromOIM` validation is a regression check for generated metadata.
54. Reverse conversion reconstructs UBL Invoice XML from structured CSV using

LHM XPath and syntax binding rules.

55. Reverse conversion adds UBL-required support values that are not EN 16931 BT

values when required for schema validity, such as `cac:TaxScheme/cbc:ID` and missing `cbc:ChargeIndicator=false` for price allowance contexts.

56. Reverse conversion normalizes supported UBL child element order before

writing XML.

57. UBL 2.1 schema validation is a regression check for regenerated XML.

Consequences

- Structured CSV output is tied directly to the xBRL-CSV taxonomy and can be

loaded by Arelle.

- The PoC distinguishes semantic values from syntax-required support values.
- Round-trip XML is not expected to be byte-for-byte identical to source XML,

but it must preserve bound values and satisfy UBL schema constraints.

- Local UBL 2.1 XSD cache remains generated or local verification data and is

not committed.

Decision 0006 Local Taxonomy Generator

Decision: Keep the Taxonomy Generator in This Repository

Context

Taxonomy generation tests previously depended on a sibling checkout of `XBRL-GL-2026`. That made the PoC harder to reproduce after moving the active GitHub synchronization target to `UADC_GIT` / `UADC-PoC`.

Decision

58. Keep the UADC-compatible taxonomy generator at

`tools/taxonomy/xBRLGL_TaxonomyGenerator.py`.

59. Keep the required `gl-gen` template at

`tools/taxonomy/gen/gl-gen-2026-MM-DD.xsd`.

60. Update tests and documentation to use the local generator.
61. Treat `../XBRL-GL-2026` as a historical/reference workspace only, not a

runtime dependency for normal PoC tests.

62. Mirror the same layout into the old `UADA/UADC_PoC` work area only for

local synchronization checks. GitHub push target remains `UADC_GIT`.

Consequences

- `tests/test_xbrlgl_generator_uadc_lhm.py` can run from a fresh clone without a

sibling `XBRL-GL-2026` checkout.

- The repository carries the generator code needed for the PoC checkpoint.
- Generated taxonomy output under `out/` remains ignored and is not committed.
- Future changes to taxonomy generation should be made in the local generator

first and then mirrored to any old working copies only when needed.